

Executive Summary

We are building an AI-powered meeting orchestration assistant that **proactively** handles scheduling, communication, and follow-ups. Unlike passive calendar tools, our assistant acts autonomously—negotiating times, sending emails, and summarizing meetings on behalf of the user. This fills a clear market need: enterprises increasingly rely on AI tools to automate meetings and collaboration. Indeed, the AI meeting assistant market is projected to surge from **\$3.67B in 2024 to \$72.17B by 2034** (CAGR $\approx$ 34.7%) ¹. This growth is driven by demand for AI-driven productivity (note-taking, summarization, scheduling) ². Our solution's unique value lies in its **active agent layer** – it not only manages availability data but **initiates** contact (via email or simulated voice), consolidates responses, and finalizes schedules. We provide two interfaces (Admin and Participant) to reflect a realistic multi-user workflow, and we emphasise robustness (logging, testing) and polish (modern UI/animations). This combination of features—unified full-stack architecture, free-tier AI/voice integration, and rigorous dev practices—will impress both investors and developers.

1. Project Overview & Unique Value Proposition

- **Goal:** Automate the *entire meeting lifecycle* (scheduling, negotiation, confirmation, follow-up) using AI.
- **USP:** Unlike standard schedulers, our assistant **actively negotiates** times and communicates with invitees. It integrates with email/SMS and eventually voice calls, giving a “Jarvis-like” personal assistant feel.
- **Market:** AI meeting assistants are high-growth (CAGR $\sim$ 34–35%) ¹. By solving complex scheduling (multi-party availability, preferences, conflicts) and handling communications, we tap into a large, enterprise-hungry market.
- **Technology Differentiators:** We use a **PostgreSQL + Prisma** backend to handle relational scheduling data with strong consistency ³. We couple this with cutting-edge UI (React+Tailwind+ShadCN+Framer) and free AI APIs (Groq/OpenRouter) to maximize capabilities without high costs.
- **Value:** Saves user time (fewer emails/calls), reduces double-booking, and provides intelligent insights (summaries, suggestions). Its investor appeal rests on strong market demand and a robust, scalable architecture that leverages both *modern AI* and *best-practice engineering*.

2. Phased Implementation Plan (Phases 1–7)

We organize development into **7 phases** over ~20–25 days. Each phase has clear deliverables, acceptance criteria, regression tests, and failure scenarios. The timeline is summarized below (details follow each phase).

```
gantt
  title Project Timeline (Phases 1-7)
  dateFormat YYYY-MM-DD
  axisFormat %b %d
  section Phase 1
  Project Setup & Foundation :done, a1, 2026-04-11, 2d
```

```
section Phase 2
DB & Core Architecture :done, a2, after a1, 2d
section Phase 3
Core Features (MVP Logic) :a3, after a2, 6d
section Phase 4
AI Integration :a4, after a3, 4d
section Phase 5
Frontend & UI :a5, after a4, 6d
section Phase 6
Simulation & Advanced Features :a6, after a5, 5d
section Phase 7
Testing, Polish & Deploy :a7, after a6, 3d
```

Phase 1: Setup & Foundation (2 days)

Goals: Establish clean project scaffolding.

- Deliverables:

- GitHub repo with `main` (protected) and `dev` branches; initial README.
- Backend: Express project initialized (ES6 modules, linting, dev server).
- Frontend: React (Vite) app initialized with Tailwind/ShadCN/Framer.
- PostgreSQL + Prisma: basic connection setup (env var, sample schema).
- Winston logging scaffold (console transport).

- Acceptance Criteria:

- `npm run dev` starts server and client on expected ports.
- Prisma connects to local Postgres (no errors).
- Logs at startup show expected messages (e.g., "Server started" via Winston).

- Regression Tests:

- Re-run after changes: server still starts, Prisma still migrates.
- Check ping routes (`GET /health`).

- Failure Scenarios:

- DB unreachable: service should log error and exit gracefully.
- Missing env vars: startup should error with clear message.

Phase 2: Database & Core Architecture (2-3 days)

Goals: Define data models and core APIs.

- Deliverables:

- **Prisma schema** with models: `User`, `Meeting`, `Participant`, `Availability`, etc., including relations. (See [Prisma docs on m:n relations ⁴].)
- Migrations: run `npx prisma migrate dev` to create tables. (Use Docker/Compose as in Prisma guide ⁵ ⁶ .)
- Express routes/controllers skeleton (e.g. `/api/auth`, `/api/meetings`, `/api/users`).
- Middleware: error handling, request logging (Winston) on each request.

- Acceptance Criteria:

- Data models reflect requirements: e.g. `Meeting` has `organizerId`, `title`, `time`, `status` .
- Relations: e.g. `Meeting` $\rightleftharpoons$ `User` via `Participant` join table (explicit many-to-many ⁴).
- Successful migration: tables created as expected (IDs, FKs).
- Basic CRUD routes respond (even with dummy data).

- Regression Tests:

- Re-run migrations: no data loss, idempotent or versioned migration.

- Verify express routes respond with correct HTTP codes for empty DB.
- **Failure Scenarios:**
- Migration conflict: handle with Prisma migrate reset or manual fix.
- API errors: return structured error JSON (use status codes 4xx/5xx).

Phase 3: Core Features (6 days)

Goals: Implement MVP scheduling logic and auth.

- Deliverables:

- **Auth:** User registration/login with JWT. Middleware to protect routes and attach `req.user` ⁷.
- **RBAC:** Roles (`admin`/`user`), enforced via middleware. E.g. admin routes checked as `if(req.user.role !== 'admin') return 403` ⁸.
- **Meeting APIs:** Admin can create meetings with title, invited users. Users can see their invites, accept/reject.
- **Availability:** Users can set available time slots (one-to-many with User or separate model).
- **Scheduling logic:** Simple conflict detection (no double-booking); possibly auto-suggest next slot if conflict.
- **Acceptance Criteria:**
- **Auth endpoints** return valid JWT (include minimal claims: user ID, role) ⁷.
- **Protected routes:** 401 for missing/invalid token, 403 for insufficient role.
- **Meeting flow:** Admin creates meeting → invited users see invite → users respond → meeting status updates appropriately.
- Database updates reflect user responses.
- **Regression Tests:**
- Test all combinations: admin vs user tokens, valid vs expired JWT.
- Schedule a meeting, restart server, data persists (Postgres).
- Simulate duplicate invites or overlapping times.
- **Failure Scenarios:**
- User not found or unauthorized: return 404/401.
- Double-booking detected: return 400 with message.
- DB errors: log via Winston and return 500.

Phase 4: AI Integration (4 days)

Goals: Add AI services for summaries and time suggestions.

- Deliverables:

- **AI Service layer:** Module to call LLM (Groq and OpenRouter) with prompt templates.
- **Email drafting:** Endpoint like `/api/ai/draft-email` that generates email body via AI (Groq/OpenRouter).
- **Meeting summary:** After meeting (or simulation step), generate summary.
- **Smart suggestions:** E.g. `/api/ai/suggest-times` that takes availability and returns proposed slots.
- **Logging:** Log all AI requests/responses for auditing.
- **Acceptance Criteria:**
- AI endpoints return plausible responses (mock free tier).
- If Groq fails, fallback to OpenRouter or vice versa.
- Respect rate limits: if exceeded, return 429 or switch model.
- **Regression Tests:**
- Mock/stub AI APIs to test integration in CI.
- Ensure error handling: if AI API times out, service returns graceful error.
- **Failure Scenarios:**

- **Rate-limit hit:** detect HTTP 429 from provider, back off or switch to alternate service.
- **Invalid response:** if AI returns nothing or gibberish, handle in code or retry.

Phase 5: Frontend & UI (6 days)

Goals: Build both Admin and User React interfaces, polished with modern UI.

- Deliverables:

- **Auth pages:** Login/Register forms, storing JWT in memory (or cookie).
- **Admin Dashboard:** Layout with sidebar (ShadCN components), meeting creation form, list of pending invites/responses.
- **User Dashboard:** List of invites, accept/reject buttons, calendar or list view.
- **API Integration:** React services call backend (using fetch or axios).
- **UI states:** Show skeleton loaders for data-fetching, error states on fetch failure.
- **Animations:** Use Framer Motion for transitions (e.g. sidebar reveal, buttons) and subtle GSAP/Framer effects for loading.
- **Styling:** Tailwind & ShadCN to implement a clean, semi-transparent “glassmorphism” design, dark/light theme toggle.
- **Acceptance Criteria:**
 - Layout matches wireframes (sidebar, content area).
 - All API calls succeed with real data (login, fetch meetings).
 - Loading states visible (skeleton screens) while awaiting API.
 - UI works on desktop and mobile widths.
- **Regression Tests (UI):**
 - Simulate no-network: show error banners.
 - Re-login user: token persists (or redirect to login).
 - Consistency: after backend restart, frontend still works with correct API base URL.
- **Failure Scenarios:**
 - **API offline:** show “Service Unavailable” banner and retry option.
 - **Unauthorized:** if 401 returned, force logout and clear state.

Phase 6: Simulation & Advanced Features (5 days)

Goals: Implement “Jarvis” simulation and advanced polish.

- Deliverables:

- **AI Assistant Panel:** Chat-style interface for admin to view AI actions (sent emails, received replies).
- **Voice Simulation:** Show fake call steps (“AI calling Alice...” with simulated transcript).
- **Email Sending:** Integrate Nodemailer (Gmail SMTP) to send real emails for invites/followups.
- **Real-time Feedback:** Use simple polling or Server-Sent Events so that user dashboard updates when admin sends invite.
- **Error Logging:** Simulate failure cases (e.g. Twilio or email API down), ensure fallback (e.g. show error in UI, retry prompts).
- **Acceptance Criteria:**
 - Emails are sent and (if using test accounts) received.
 - The AI panel logs each step (with timestamps).
 - User sees updates without manual refresh (within ~5 seconds).
- **Regression Tests:**
 - Test full cycle: Admin creates meeting → AI panel shows steps → user receives email or sees invite → user responds → admin sees confirmation.
 - Turn off email/SMS service: ensure system handles it (e.g. logs error, doesn't crash).
- **Failure Scenarios:**

- **Nodemailer fails:** catch and log error, notify admin in UI.
- **Call simulation:** If STT/TTS fails, show fallback (e.g. just text script).

Phase 7: Testing, Polish & Deployment (3 days)

Goals: Final QA, documentation, and prep for launch.

- Deliverables:

- **Jest Test Suite:** Unit tests for services (e.g. meeting logic), and integration tests (with Supertest) for APIs ⁹. Cover at least 80% paths (auth, meeting creation, etc.).
- **CI Workflow:** GitHub Actions YAML: linting, tests on push to `dev`, and optional deployment to staging.
- **Logging Setup:** Winston to also log to file (with rotation).
- **Monitor Stub:** Simulate alert (e.g. health-check script, or console log on error).
- **Docker Compose:** `docker-compose.yml` for Postgres+Prisma (per [Prisma Docker guide ⁵]). Seed script (e.g. `npx prisma db seed`).
- **Production Notes:** E.g. environment config for Heroku/Vercel.
- **Documentation:** README with setup instructions, key endpoints, and developer notes.
- **Acceptance Criteria:**
 - All Jest tests pass. CI pipeline passing on GitHub.
 - Local deployment works via Docker Compose.
 - README clearly explains how to run and use the app.
- **Failure Scenarios:**
 - **Broken commit:** CI catches any failing test or lint; fix before merge.
 - **Production errors:** Ensure uncaught exceptions are caught by Winston (or logged) to avoid crashes.

3. Technical Architecture Diagrams

```
erDiagram
    USER ||--o{ PARTICIPANT : attends
    MEETING ||--o{ PARTICIPANT : includes
    USER ||--o{ MEETING : organizes
    PARTICIPANT {
        int meetingId
        int userId
        enum status
    }
    MEETING {
        int id
        string title
        datetime time
        int organizerId
    }
    USER {
        int id
        string name
        string email
        string role
    }
```

```
sequenceDiagram
    participant Admin
    participant API
    participant AI
    participant User
    Admin->>API: Create meeting & invite (JWT auth)
    API->>DB: Store meeting + invites
    API->>AI: "Generate email text"
    AI-->>API: Return email body
    API->>User: Send invitation (email/SMS)
    User-->>API: Accept or reject invite
    API->>DB: Update participant status
    API-->>Admin: Notify of final schedule
```

```
flowchart LR
    Start[Start Scheduling] --> AuthCheck{"User Authenticated?"}
    AuthCheck -- No --> RedirectLogin[/Show Login Page/]
    AuthCheck -- Yes --> ShowDashboard[Display Dashboard]
    ShowDashboard --> Action{Action Type}
    Action -- "Create Meeting" --> CreateForm[Show Meeting Form]
    Action -- "View Invites" --> ListInvites[Show Invites List]
    CreateForm --> SubmitMeeting[/POST /api/meetings/ to backend/]
    SubmitMeeting --> AIFlow[Call AI Services (email, time)]
    AIFlow --> NotifyParticipants[Send Emails/Notifications]
    NotifyParticipants --> UpdateUI[Refresh Admin Dashboard]
    ListInvites --> RespondInvite[/Accept/Reject POST to /api/participants/]
    RespondInvite --> UpdateUI
    UpdateUI --> End[End Flow]
```

4. Database (Prisma Schema & Migrations)

We use **PostgreSQL** with **Prisma ORM** for a strong relational schema ³. Core models include `User`, `Meeting`, and a join table `Participant` for the many-to-many relation. A possible Prisma schema excerpt:

```
model User {
  id          Int          @id @default(autoincrement())
  name        String
  email        String       @unique
  password     String
  role         Role         @default(USER)    // e.g. ADMIN, USER
  meetings     Meeting[]    @relation("Organizer")
  invites      Participant[]
  createdAt    DateTime     @default(now())
}

model Meeting {
```

```

    id          Int          @id @default(autoincrement())
    title       String
    time        DateTime
    organizer   User          @relation("Organizer", fields: [organizerId],
references: [id])
    organizerId Int
    participants Participant[]
    status      MeetingStatus @default(PENDING)
    createdAt   DateTime      @default(now())
  }

  model Participant {
    meeting Meeting @relation(fields: [meetingId], references: [id])
    meetingId Int
    user      User    @relation(fields: [userId], references: [id])
    userId    Int
    status    String // e.g. PENDING, ACCEPTED, REJECTED
    @@id([meetingId, userId])
  }

  enum Role {
    ADMIN
    USER
  }

  enum MeetingStatus {
    PENDING
    CONFIRMED
    CANCELED
  }

```

- **Relationships:** Meeting → User (organizer) is one-to-many. Meeting ↔ User via Participant is many-to-many ⁴.
- **Migrations:** After defining the schema, run `npx prisma migrate dev --name init` to generate SQL (as per official guide ⁶). Example Docker Compose (Prisma docs) uses environment vars and healthcheck for Postgres ⁵.
- **Indexes & Constraints:** Primary keys (`@@id` in Participant), foreign keys on relations (Prisma adds them automatically). Add unique index on (`User.email`).
- **Sample Migration Steps:** (From [41])
 1. Create `docker-compose.yml` with Postgres service and env (e.g. `POSTGRES_USER=postgres`, `POSTGRES_PASSWORD=prisma`) ⁵.
 2. Run `docker compose up -d` and set `DATABASE_URL` in `.env`.
 3. Run `npx prisma migrate dev --name init` to create tables (creates `prisma/migrations`).
 4. (Optional) `npx prisma db seed` for initial data.

5. API Design

All APIs under `/api`. Use REST style with JSON. Validate requests with **Zod** schemas ¹⁰ ¹¹. Example endpoint breakdown:

- **Auth (no auth needed):**
 - `POST /api/auth/register` - Request: `{ name, email, password }`; Response: `201 Created` + user data. Validate via Zod.
 - `POST /api/auth/login` - Request: `{ email, password }`; Response: `200 OK` + `{ token: JWT }`.
 - Use `jsonwebtoken` on backend; return 401 if invalid.
- **User (protected):**
 - `GET /api/users/me` - Return current user (from JWT).
 - `GET /api/users/:id` - (admin) fetch any user.
- **Meetings:** (auth required)
 - `POST /api/meetings` - Create meeting (admin only). Body: `{ title, time, participantIds: [Int] }`. Response: `201` with meeting. Validate with Zod.
 - `GET /api/meetings` - List all for admin, or for user only their meetings/invites.
 - `GET /api/meetings/:id` - Detail.
 - `PATCH /api/meetings/:id` - Update (e.g. reschedule or cancel). Admin only.
 - `DELETE /api/meetings/:id` - Cancel meeting (admin only).
- **Participants (invites):** (protected)
 - `PATCH /api/participants/:meetingId/respond` - Body: `{ status: "ACCEPTED" | "REJECTED" }`. Response: `200` and updated status.
- **AI Services:** (protected admin)
 - `GET /api/ai/suggest-times` - Query suggested times based on availability (calls Groq/OpenRouter).
 - `POST /api/ai/draft-email` - Body: `{ meetingId, context }`, returns email body text.
 - `POST /api/ai/summary` - Body: `{ meetingId, notes: String }`, returns summary.

Request/Response Schemas: Define with Zod for type safety and validation ¹² ¹¹. Example Zod schema for meeting creation:

```
const meetingCreateSchema = z.object({
  title: z.string().min(1),
  time: z.string().refine(isISODate), // or use a custom validation
  participantIds: z.array(z.number().int()).min(1),
});
```

Each route uses a middleware like:

```
function validate(schema) {
  return (req, res, next) => {
    try { schema.parse(req.body); next(); }
    catch(e) { res.status(400).json({ error: e.errors }); }
  };
}
```


Authentication & RBAC: Use JWT in `Authorization: Bearer <token>`. Middleware verifies token, attaches decoded payload (including user ID and role) to `req.user` ⁷. Example:

```
app.use((req,res,next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({error:"No token"});
  try {
    const payload = jwt.verify(token, JWT_SECRET);
    req.user = payload;
    next();
  } catch { return res.status(401).json({error:"Invalid token"}); }
});
```

For RBAC, check `req.user.role`. E.g. admin check:

```
if (req.user.role !== 'ADMIN') {
  return res.status(403).json({error: 'Admin access required'});
}
```

Error Codes: Use standard HTTP codes: - `200 OK` on success,

- `201 Created` on new resource,
- `400 Bad Request` for validation errors,
- `401 Unauthorized` for missing/invalid token,
- `403 Forbidden` for insufficient role or token mismatch ¹⁴,
- `404 Not Found` if resource (user/meeting) doesn't exist,
- `429 Too Many Requests` if AI rate-limit is hit,
- `500 Internal Server Error` for unhandled issues (log these with Winston).

Structured Errors: Always return `{ status:"error", message:"..."}` or a list of field errors to keep API consistent.

6. Frontend Architecture & UI

Folder Layout

```
src/
  api/           # API call functions (wrapped fetch/axios)
  assets/        # images/fonts
  components/
    common/      # buttons, forms, layout
    Admin/       # admin-specific pages
    User/        # user-specific pages
```

```

    AI/           # AI assistant panel
    hooks/        # custom React hooks (e.g. useAuth)
    context/      # AuthContext for current user
    pages/        # Login, Dashboard, etc.
    routes/       # React Router setup (Admin vs User routes)
    utils/        # helpers (date formatting, etc.)
    App.jsx
    main.jsx

```

- **Admin vs User Interface:** Use role from token to render correct routes. E.g. `/admin/*` for admin pages, `/` for user.
- **Key Components:**
 - **Sidebar/Nav:** ShadCN `Sidebar` with menu.
 - **Header:** shows user info, logout.
 - **MeetingList:** table or cards of meetings.
 - **MeetingForm:** form to create/edit meetings (inputs for title, time-picker, multi-select of users).
 - **InviteList:** for users to accept/reject. Buttons should use Framer's `<motion.button>` for hover effects.
 - **AIChatPanel:** scrollable chat window showing AI actions (styled like Slack or Telegram).
 - **Skeletons:** Use ShadCN's skeleton components (or Tailwind skeleton classes) to show loading for lists and forms.
- **UI States:**
 - **Loading:** skeleton screens in place of data until API returns.
 - **Empty:** friendly message/illustration if no meetings or invites.
 - **Error:** banner/toast on network/API error.
- **Animations:**
 - **Framer Motion:** page transitions (fade in pages, slide menu in/out), list item hover animations.
 - **GSAP (optional):** for a fancier loader (e.g. animated dots) if needed.
- **Design Notes:** Use Tailwind's utility classes for quick styling. Employ glassmorphism: `bg-white/20 backdrop-blur-lg` for panels ¹⁵. Ensure high contrast text for accessibility.
- **Figma/Stitch:** Though code-first, maintain a simple Figma mockup of key screens (Admin dashboard, user panel). ShadCN's theme generator can help establish color palette.

7. AI Integration Plan

We leverage **Groq** and **OpenRouter** for LLM tasks (both offer free tiers) ¹⁶ ¹⁷. Key considerations:

- **Prompt Templates:** Store template prompts for meetings (e.g. "Draft a polite email inviting participants to a meeting at [time] and summarizing the agenda."). Keep them in code with placeholders.
- **LLM Usage:** Abstract calls via a service layer (e.g. `aiService.callLLM(model, prompt)`) that handles HTTP requests. Prefer Groq by default (higher free rate: 1000 req/day ¹⁶) and fallback to OpenRouter. If Groq returns error or limit reached, switch.
- **Fallbacks:** Both have rate limits: OpenRouter free models allow 20 RPM, 200/day ¹⁷; Groq allows 1000/day, 6000 tokens/minute ¹⁶. Code should catch 429 from one provider and retry on the other or queue request.
- **Logging:** Use Winston to log each AI request/response (truncate in logs to avoid huge text). For example:

```
logger.info("AI Request", { model, prompt });
const res = await groqClient.chat.completions.create(...);
logger.info("AI Response", { content: res.choices[0].message.content });
```

- **No-cost Usage:** We intentionally use their free endpoints (e.g. model IDs ending in `:free` for OpenRouter). Monitor usage. In presentations, note that these are pay-as-you-go so costs are minimal (Groq has free dev tier up to limits ¹⁸).
- **Rate Limit Monitoring:** Before each call, optionally check remaining quota (OpenRouter API offers key info ¹⁹) to preempt exhaustion.

Prompt Examples

- **Meeting summary:** "Summarize the following meeting transcript into bullet points and action items: {transcript}"
- **Email draft:** "Write a professional invitation email for a meeting titled '{title}' scheduled at {time}, inviting {participants} to respond."
- **Time suggestion:** "Given these free times for users: {availabilities}, suggest three optimal time slots for a 1-hour meeting."

These will be embedded into the AI service calls.

8. Voice Strategy

Phase 1 (MVP): We implement **simulated voice interactions** rather than real calls, to keep things reliable. The flow:

- When admin opts for "Call participant", UI shows a mock dialog:
- "Calling {User Name}..."
- After a pause, display `User: "I can meet at 3pm."` (simulated response).
- Behind scenes, this uses the same AI prompt engine to generate a plausible script (e.g. using Groq to produce both sides of dialogue). No actual STT/TTS is used yet. This looks like a voice call in UI without telephony integration.

Phase 2 (Interactive browser-based voice):

- Explore Web Speech API for TTS/STT ¹⁵. For example, use `window.speechSynthesis.speak()` to read AI text, and `SpeechRecognition` for user voice (if we want live demo). This adds real audio input/output but stays in-browser (no phone call). Useful for on-device testing.

Phase 3 (Real Twilio integration):

- Use Twilio Programmable Voice to place actual calls ²⁰. Steps:
 1. Acquire Twilio number (free trial available) ²¹.
 2. Use Twilio Node SDK to create calls:

```
const client = require('twilio')(TWILIO_SID, TWILIO_TOKEN);
client.calls
  .create({ url: 'http://your-server/twiml', to: userPhone, from:
    twilioNumber });
```

3. Host a `/twiml` endpoint that returns TwiML (XML) instructions for the call (e.g. say "Hello, I'm the AI assistant...").

4. Handle voice input: use Twilio's Webhooks for speech (TwiML `<Gather>` tag or Twilio Speech Recognition).

- **Sequence (Phase 3):**

Admin triggers call → Server uses Twilio SDK to dial user → Twilio requests `/twiml` and synthesizes speech → User speaks → Twilio sends transcription back to our server → Server processes response.

- **Failure Handling:**

- If Twilio API errors (e.g. rate-limit or connectivity), fallback to email/SMS.

- Log failures (Twilio will post errors to your webhook as well).

9. Logging & Monitoring

We use **Winston** for structured logging ²² ²³. Configuration:

```
import winston from 'winston';
const logger = winston.createLogger({
  level: 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.json()
  ),
  transports: [
    new winston.transports.Console(),
    new winston.transports.File({ filename: 'combined.log' })
  ]
});
```

- **Log Levels:** `error`, `warn`, `info`, `debug`. E.g., log incoming requests at `info`, errors at `error`.

- **Formats:** JSON with timestamp (facilitates analysis) ²⁴.

- **Transports:** Console for dev; file (with rotation via `max-size` config) for persistence ²⁵.

- **Retention:** Set `max-size: "10m"`, `max-file: 3` on file transport to keep logs bounded.

- **Structured Logging:** For each HTTP request, log: `{ route, method, userId }`. On errors, include stack trace.

- **Alerting:** In production, connect logs to an external monitor (e.g. Loggly/Datadog) via Winston transport. Locally, simulate by checking error logs manually.

- **Simulated Failures:** During testing, deliberately trigger errors (e.g. wrong DB credentials) and verify:

- Application logs the error with stack at `error` level.

- If error is critical (DB down), process exits or restarts gracefully.

- Health-check route (`GET /health`) should report status ("OK" or "ERROR") for liveness monitoring.

10. Testing Strategy

• **Unit Tests (Jest):**

• Test pure functions: date utilities, scheduling conflict logic, email template generation.

• Test controllers with mocked services (e.g. mock Prisma calls).

• **Integration Tests (Jest + Supertest):**

- Use **Supertest** to test Express routes ⁹. Example:

```
const response = await request(app)
  .post("/api/meetings")
  .set("Authorization", `Bearer ${token}`)
  .send({ title:"Test", time:"2026-04-12T10:00", participantIds:[2,
3] });
expect(response.status).toBe(201);
expect(response.body).toHaveProperty("meeting");
```

⁹

- Cover all API endpoints (auth, users, meetings, participants).
- **Test Data:** Create a test database (e.g. SQLite in-memory or a separate Postgres DB) for CI. Use `jest --runInBand` with a fresh schema or use **Prisma Test** tooling.
- **Regression Checklist (Dev Tests):** After each change, verify:
 - **Auth:** token issuance and validation.
 - **Meeting Flow:** creation → user response → status update.
 - **Error Cases:** bad JSON (expect 400), invalid token (401), forbidden access (403).
 - **Boundary:** large inputs (long title), invalid dates.
- **CI (GitHub Actions):**
- Run on each push/PR:
 - `npm run lint` (ESLint/Prettier),
 - `npm test` (Jest),
 - Build front-end (Vite).
- Optionally deploy staging on `dev` branch via Render or Heroku.
- **Manual QA:** Test on multiple browsers (Chrome, Edge, mobile). Check feature toggles (enable/disable AI).

11. Dev Workflow & Git Discipline

- **Branches:** `main` (production), `dev` (integration). Feature branches for big features.
- **Commit Messages:** Follow Conventional Commits format (type: scope – short) as earlier decided. E.g.
- `feat(auth): implement JWT login`
- `fix(meeting): handle conflict detection properly`
- **Example Commands:**
- Initialize repo: `git init && git checkout -b dev`
- After each completed task:

```
git add .
git commit -m "feat: <description>"
git push origin dev
```

- **Initial Setup Commits:**
- `chore: initialize git repo, Node and React projects`
- `feat: configure Express server and Prisma setup`
- **Sample History (Final):**

```

feat: setup frontend (Vite) and basic pages
feat: add Express server and middleware
feat: define Prisma schema (User, Meeting, Participant)
feat: implement user authentication (JWT)
feat: create meeting and invite APIs
feat: add meeting confirmation logic
feat: integrate Groq/OpenRouter AI services
feat: build admin dashboard UI with Tailwind & ShadCN
chore: add Jest tests and CI pipeline

```

- **Pull Requests:** Keep simple (no massive merges). For this project, minimal PR overhead—focus on clean commits as each task.

12. Deployment & Local Setup

- **Local:** Use **Docker Compose** for Postgres. Example from Prisma guides ⁵:

```

version: '3.7'
services:
  postgres:
    image: postgres:15
    environment:
      - POSTGRES_DB=postgres
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=prisma
    ports: ["5432:5432"]

```

Then `docker-compose up -d`. Set `DATABASE_URL` in `.env` accordingly.

- **Seeding:** Use `npx prisma db seed` if seeds defined, or manually insert an admin user (bcrypt-hashed) via `npx prisma studio`.
- **Dockerize App:** Create a `Dockerfile` (multi-stage) per [Prisma docs ²⁶]: use `node:alpine` or `node:slim`, copy code, `npm ci`, and CMD to run migrations & start. E.g.:

```

FROM node:lts-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
CMD ["sh", "-c", "npx prisma migrate deploy && node server.js"]

```

- **Production:**
- **Backend:** Can deploy on Heroku/Render. Use environment vars (JWT secret, DB URL). Heroku needs `web: node server.js` in `Procfile`.
- **Frontend:** Can deploy on Vercel/Netlify as static site; set `REACT_APP_API_URL` to deployed backend.
- **Database:** Use cloud Postgres (Heroku Postgres, AWS RDS, Supabase (though chosen Postgres self-managed)).

- **Env Vars:** `JWT_SECRET`, `DATABASE_URL`, `SENDGRID_API_KEY` (if using email service), `TWILIO_SID/TOKEN` for Phase 3. Store in `.env` (local) and in deployment platform's config.
- **Monitoring:** In production, check `GET /health` for status, and logs via hosting platform.

13. Documentation & Investor Materials

- **README (GitHub):** Sections: Project Intro, Tech Stack, Setup Instructions (prerequisites, `npm install`, `docker-compose up`, `npm run dev`), API docs (list endpoints), key features (with screenshots), How to Contribute.
- **Investors Slides:**
 - **Title:** AI Executive Assistant (logo)
 - **Problem:** Difficulty/time-consuming scheduling and coordination.
 - **Solution:** Our AI-driven assistant (high-level diagram).
 - **Features:** Active scheduling, voice/email interface, summaries (bullets).
 - **Market:** Growth stats (use [36] figures).
 - **Tech Stack:** Modern full-stack, highlight scalability.
 - **Roadmap:** Phase 1–3 bullet points (MVP vs future).
 - **Team/Owner:** (You) + advisors (if any).
 - **Ask:** (resources needed) or next steps.
 - **1-Minute Pitch:**

“Our product is an AI meeting assistant that autonomously schedules and manages your meetings. It finds times, negotiates with participants, sends emails, and even provides summaries — all with minimal user effort. By combining a relational backend (Postgres/Prisma) with cutting-edge LLM APIs (Groq/OpenRouter) and a polished React frontend, we deliver a robust, real-world solution. The AI meeting assistant market is booming (projected \$72B by 2034 ¹), and our system targets both individual professionals and enterprises to drastically cut coordination overhead. In short, it's like having your own digital “Jarvis” for meetings.”

14. Comprehensive Checklist

- ☐ **Tech Stack Finalized:** React/Vite, Tailwind, ShadCN, Framer; Node/Express; PostgreSQL + Prisma ³.
- ☐ **Git Repo:** Initialized with `main` / `dev`, `.gitignore`, ESLint, Prettier.
- ☐ **Phase 1 Completed:** Express + React set up, logging configured (Winston console).
- ☐ **Phase 2 Completed:** Prisma schema coded (User, Meeting, Participant); migrations run ⁶; basic controllers/routes scaffolded.
- ☐ **Phase 3 Completed:** Auth endpoints (JWT with `req.user`) ⁷; RBAC enforced; meeting APIs working.
- ☐ **Phase 4 Completed:** AI service module calling Groq/OpenRouter; email draft/summary endpoints; error handling on rate-limit.
- ☐ **Phase 5 Completed:** Admin and User UIs built with ShadCN/Tailwind; API integration works; skeleton loaders in place.
- ☐ **Phase 6 Completed:** AI Assistant chat panel built; Nodemailer integrated; real-time updates; voice call simulation UI implemented.
- ☐ **Phase 7 Completed:** Jest tests (unit + integration) written ⁹; CI workflow set up; Docker Compose for Postgres ⁵; README and documentation finalized.
- ☐ **Future (Phase 8):** Real Twilio voice calls; further AI refinement; performance tuning.

Each checkbox corresponds to tasks detailed above, ensuring nothing is missed.

Sources: Relevant documentation and best practices guides were used, including the Prisma schema guide ⁴ , JWT usage recommendations ⁷ , Winston logging tutorials ²³ , and AI API docs ¹⁷ ¹⁶ .

¹ ² AI Meeting Assistant Market Size, Share | CAGR at 34%

<https://market.us/report/ai-meeting-assistant-market/>

³ MongoDB vs PostgreSQL - Difference Between Databases - AWS

<https://aws.amazon.com/compare/the-difference-between-mongodb-and-postgresql/>

⁴ Many-to-many relations | Prisma Documentation

<https://www.prisma.io/docs/v6/orm/prisma-schema/data-model/relations/many-to-many-relations>

⁵ ⁶ ²⁶ How to use Prisma in Docker | Prisma Documentation

<https://www.prisma.io/docs/guides/deployment/docker>

⁷ ⁸ ¹⁴ How To Use JSON Web Tokens (JWTs) in Express.js | DigitalOcean

<https://www.digitalocean.com/community/tutorials/nodejs-jwt-expressjs>

⁹ Guide to writing integration tests in express js with Jest and Supertest - DEV Community

https://dev.to/ali_adeku/guide-to-writing-integration-tests-in-express-js-with-jest-and-supertest-1059

¹⁰ ¹¹ ¹² ¹³ Validating Request Data in Express.js using Zod and TypeScript: A Comprehensive Guide - DEV Community

<https://dev.to/osalumense/validating-request-data-in-expressjs-using-zod-a-comprehensive-guide-3a0j>

¹⁵ Using the Web Speech API - Web APIs | MDN

https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API/Using_the_Web_Speech_API

¹⁶ ¹⁷ 15 Free LLM APIs You Can Use in 2026

<https://www.analyticsvidhya.com/blog/2026/01/top-free-llm-apis/>

¹⁸ Is there a free tier and what are its limits? - FAQs - Groq Community

<https://community.groq.com/t/is-there-a-free-tier-and-what-are-its-limits/790>

¹⁹ API Rate Limits | Configure Usage Limits in OpenRouter | OpenRouter | Documentation

<https://openrouter.ai/docs/api/reference/limits>

²⁰ ²¹ How to Make a Phone Call Using Node.js and Twilio Programmable Voice | Twilio

<https://www.twilio.com/en-us/blog/make-phone-call-using-node-js-twilio-programmable-voice>

²² ²³ ²⁴ ²⁵ Error Handling and Logging in Node.js with Winston - Galaxy Blog

<https://blog.galaxycloud.app/error-handling-and-logging-in-node-js-with-winston/>